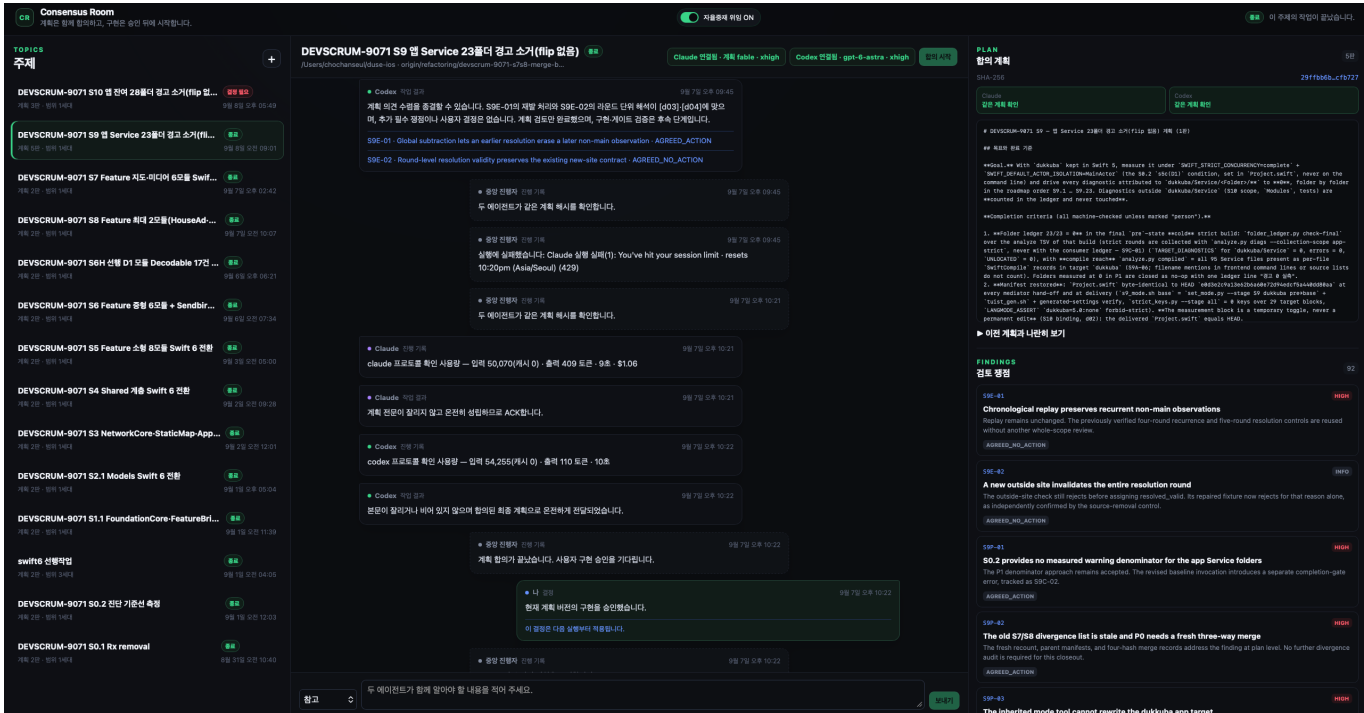


01. Claude·Codex 공동 개발 작업 자동화



실제 운영 화면: 주제 목록, 대화·결정 기록, 합의 계획과 검토 쟁점

프로젝트 시작 배경

Codex(GPT 5.6 Sol)는 계획 수립과 코드 검토에, Claude(Opus 5 또는 Fable 5)는 코드 작성 속도와 완성도에 강점이 있었습니다. 특히 Claude의 계획과 구현 결과를 Codex가 검토하면, 한 세션에서 놓친 문제를 다른 세션에서 발견하는 경우가 있었습니다.

처음에는 두 세션의 답변을 직접 복사해 전달했습니다. “Claude가 이렇게 말하는데 어떻게 생각해?”라고 Codex에 묻고, 그 답변을 다시 Claude에 보내 검토를 요청했습니다. 검토를 반복해 두 세션의 의견이 수렴했을 때는 근거 없는 답변이 줄고 작업 결과도 좋아졌습니다.

리팩토링이 길어질수록 계획, 코드 변경, 리뷰 결과를 매번 복사해 전달하기가 번거로웠습니다. 최신 계획과 앞선 지적 사항을 직접 대조해야 했고, 검토가 끝날 때마다 다음 작업도 요청해야 했습니다. 이 과정을 자동화하려고 두 모델이 서로의 계획과 구현 결과에서 오류와 누락을 찾고, 수정과 재검토를 반복하는 Consensus Room을 만들었습니다.

시스템 개요

앱은 계획부터 구현·리뷰·Git 전달까지 기록합니다. 서버는 모델 응답을 검사하고 승인된 계획과 변경 파일을 해시로 연결합니다. Claude 중재 세션은 검증 실행·결정 전달·복구·사용량 비교·시스템 유지보수를 맡습니다. React UI에서는 진행 상황을 확인하고 자율 중재 위임을 켜고 끌 수 있습니다.

구현 환경: TypeScript · Node.js · Fastify · React · SQLite WAL · Zod · Git worktree · Claude/Codex CLI

GitHub github.com/ChanSeul/consensus-room

02. 운영 화면과 주요 개선

계획 확인과 사용자 승인, 반복 작업을 줄인 변경



확대 화면: 모델의 계획 확인, 토큰별 사용량, 사용자의 구현 승인

주요 개선

문제	개선 내용
대화·계획·리뷰를 파일로 전달하고 진행 상태를 직접 확인	웹 앱에서 대화와 계획 승인·쟁점 추적·Git 전달 상태를 관리
동시 실행 충돌, 남은 자식 프로세스, 저장 파일 변조	문제를 재현해 실행 기록·파일 검증을 보강하고 엔진의 역할을 분리
긴 계획을 매번 출력하고 구현 세션에 탐색 이력 누적	계획은 부분 수정하고, 구현은 계획 탐색 이력 없이 새 세션에서 시작
결정 대기로 멈춘 작업과 수정 후 리뷰에서 반복 발생	중단 결과와 리뷰 세션을 재사용하고 변경 코드만 전달, 주제별 설정 분리
범위 밖 수정·한도 오류 처리와 실행 시간 구분 필요	변경 규칙 검사·재시도 예약, 도구 실행 시간과 모델 호출 시간을 분리

03. 시스템 구성

UI·API·CLI·저장소 연결 구조



04. Claude 중재 세션의 운영 역할

작업 준비·검증 실행·결정 근거 확인·실행 복구

Claude 중재 세션은 사용자가 방을 운영하는 일을 대행합니다. 주제를 준비하고 진행 상태를 확인하며, 계획에서 맡긴 검증을 실행하고 결과를 게시합니다. 서버가 호출하는 Claude 러너는 계획·구현·수정을, Codex는 감사·리뷰를 담당하고, 중재 세션은 이들이 다음 작업을 이어 갈 수 있도록 외부 실행과 사용자 결정을 연결합니다.

단계마다 중재 세션이 하는 일

시점	중재 세션의 작업
작업 시작 전	승인된 로드맵에서 작업 범위와 시작 조건을 정리해 주제를 만듭니다. 참여 모델과 작업 폴더를 연결하고, 이전 단계의 도구·검증 기록을 넘깁니다. 계획 전에 도구 자기검사를 실행해 실제 실패 결과도 함께 전달합니다.
계획·구현 진행 중	서버의 상태와 결정 요청을 확인합니다. 현재 계획의 규칙, 앞선 사용자 결정, 운영 기록, 코드와 실행 결과를 대조합니다. 위임으로 판단할 수 있는지 확인하고, 사용자 선택이 필요한 요청은 그대로 전달합니다.
외부 검증이 필요할 때	계획이 중재자에게 맡긴 빌드·측정을 실행하고, 종료 코드·로그·측정값을 증거로 게시합니다. 화면 확인이 필요한 경우 스크린샷을 전달합니다. 사용자 결정이나 필요한 증거가 갖춰지면 작업을 재개합니다.
작업 완료·다음 단계	검증과 리뷰 결과를 확인한 뒤 허용된 승인·커밋·push·종료를 요청합니다. 완료한 범위, 남은 쟁점과 다음 시작 조건을 기록해 다른 세션이 이어받게 합니다. 범위 밖 후속 항목의 처분은 사용자에게 묻습니다.

실제 적용: Swift 6 전환 검증

전환 작업에서는 중재 세션이 새 작업 폴더에서 승계한 검증 도구를 먼저 실행하고, 실패한 항목을 계획 시작 자료에 넣었습니다. 구현 뒤에는 계획에 정한 xcodebuild·측정·종료 검사를 실행해 근거를 게시했습니다. 화면 잠금 때문에 UI 확인이 막힌 경우에는 해당 검증을 남겨 두고, 그 결과에 의존하지 않는 후속 작업을 진행했습니다.

결정문은 기존 계약과 실행 근거에서 작성

중재 세션은 계획에 이미 정한 조건과 이전 사용자 결정으로 해결할 수 있는 요청인지 먼저 확인합니다. 위임 범위에서 대신 내린 결정도 근거와 함께 사용자에게 보고하고, 타임라인에는 user/decision으로 기록합니다. 새 설계 선택이나 범위 변경이 필요하면 사용자의 판단을 받아 다음 실행에 전달합니다.

사용자 → 위임 범위와 중요한 선택 결정
 중재 세션 → 준비·검증·결정 전달·복구·인수인계
 서버 → 상태·계획 해시·승인·응답·변경 파일 검사
 Claude 러너 / Codex → 계획·구현·수정 / 감사·리뷰

05. 자율 중재 위임과 상시 운영

사용자 결정 대행과 시스템 유지보수의 범위 구분

중재 세션에 어디까지 맡겼는지 분명히 하기 위해 자율 중재 위임을 ON/OFF로 바꿀 수 있게 했습니다. 웹 화면과 셀 명령은 같은 설정 파일을 사용하며, 변경 시각과 이력을 저장합니다. 중재 세션은 결정할 때마다 설정을 다시 읽고, 파일이 없으면 OFF로 취급합니다.

구분	위임 ON	위임 OFF
결정 요청	승인된 계획·선례·위임 범위 안에서 근거를 붙여 결정하고 보고합니다.	요청 본문과 선택지를 원문 그대로 보여 줍니다. 사용자 답을 받은 뒤 그 내용을 결정으로 게시합니다.
개발 작업 승인·전달	명시적으로 위임받은 범위의 승인·구현 시작·커밋·push·종료를 진행합니다.	승인·구현 시작·커밋·push·종료는 사용자 확인 후 진행합니다.
검증·실행 복구	계획에 정한 측정, 증거 게시, 조건 충족 후 재개와 비용 낭비를 막는 중단을 수행합니다.	계획에 정한 측정과 증거 게시를 계속합니다. 사용자 결정 게시 뒤 재개하고, 비용 방어용 중단·상태 보고·인수인계를 맡습니다.

위임을 꺼도 계속하는 효율 개선

사용자는 결정 대행과 별개로 시스템 효율을 계속 개선할 권한을 부여했습니다. 중재 세션은 단계·역할별 입력·캐시·출력 토큰, 실행 시간과 비용을 비교합니다. 한 번에 한 가지를 바꾸고 같은 조건의 작업끼리 전후를 대조하며, 효과가 없거나 나빠지면 되돌리도록 운영했습니다.

예를 들어 리뷰 전용 세션을 도입한 뒤에도 사용량이 늘어난 표본이 있어, 세션 분리 자체를 비용 절감으로 판단하지 않았습니다. 첫 코드 리뷰와 수정 뒤 최종 리뷰를 구분하고, 비교할 코드 범위와 회차를 맞춰 다음 개선을 판단했습니다.

Claude·Codex가 작업을 이어 가도록 유지보수

중재 세션은 실행·재개 실패, 세션 식별값, 모델 설정, 상태 조회 스크립트, API 호출과 DB 기록을 점검합니다. 운영 중 발견한 결함을 고치고 검증하며, 실행 중인 턴을 끊지 않도록 비어 있는 시점에 서버를 재시작하는 규칙을 두었습니다. 변경 내용과 결과는 다음 세션이 읽을 수 있도록 기록합니다.

위임과 관계없이 사용자가 결정하는 경계

새로운 범위·계약 변경, 선례가 없는 중요한 설계 선택, 되돌리기 어려운 전달과 사용자가 직접 해야 하는 인증은 사용자에게 넘깁니다. 실제 Swift 6 전환에서는 작업 브랜치 전달만 위임했고, 개발 브랜치로의 병합과 릴리스·프로덕션 브랜치 반영은 제외했습니다. 범위 밖 후속 항목도 전달 전에 사용자에게 처분을 묻습니다.

위임 스위치는 중재 세션의 결정 대행 범위를 정하는 운영 설정입니다. 서버의 상태·승인 검사는 별도로 동작합니다. 유지보수 권한도 원격 push까지 자동으로 포함하지 않으며, OFF 상태에서 승인·상태 전이·증거 판정 규칙을 바꾸려면 사용자 확인을 받습니다.

06. 요청 처리와 데이터 저장

HTTP 요청·모델 실행·저장 데이터

구현 요청 처리

순서	검사·실행 내용
1. 요청 접수	Fastify가 인증과 요청 형식을 검사합니다. Idempotency-Key와 요청 본문을 실행 요청 기록에 먼저 등록해 재전송을 구분합니다.
2. 승인 검사	현재 상태, 양쪽 모델의 계획 확인값, 사용자 승인 해시를 대조합니다. 진행 중인 실행이 있으면 새 작업을 시작하지 않습니다.
3. 실행 등록	action을 기록하고 취소 신호를 연결합니다. 구현 브랜치와 시작 HEAD를 확인한 뒤 Claude 구현 세션을 만들거나 이어 씁니다.
4. 결과 확인	CLI 결과의 JSON 형식, 응답 종류, 지적 ID와 처분을 검사합니다. 실행 시작 시점의 세대·상태와 현재 값을 대조하고 취소 여부를 확인합니다.
5. 다음 단계	구현 보고를 저장하고 변경 범위를 검사한 뒤 Codex 리뷰를 실행합니다. 진행 이벤트는 SQLite에 남기고 SSE로 화면에 보냅니다.

저장 테이블과 사용처

저장 대상	핵심 값과 사용처
topics	현재 상태, 범위 세대, 계획 회차·해시, 승인 해시, 재개 상태, 구현 세션, 리뷰본과 Git OID를 보관합니다. 다음 동작을 결정할 때 참조합니다.
participants / review sessions	계획 참여자의 세션과 ACK를 저장합니다. 코드 리뷰 세션은 별도 테이블에 세대·계획 회차·계획 해시와 함께 기록합니다.
actions / requests	실행의 진행·완료·취소 상태와 PID·프로세스 그룹을 저장합니다. 요청 키·본문·응답은 별도로 남겨 중복 호출과 결과 미확정을 구분합니다.
timeline_events	주제별로 증가하는 sequence로 메시지와 상태 변경을 정렬합니다. 사용자 결정·증거와 실행 결과를 하나의 타임라인에서 확인합니다.
artifacts	계획·리뷰·구현 보고의 종류, 버전, 세대, SHA-256과 파일 경로를 저장합니다. 본문은 내용 해시로 이름 붙인 파일에 둡니다.

SQLite는 WAL 모드와 외래 키를 사용합니다. 상태 변경과 관련 이벤트는 같은 트랜잭션에 반영하되, 모델 호출을 기다리는 동안에는 DB 트랜잭션을 유지하지 않습니다. 호출 중에 중단이나 범위 변경 요청이 들어올 수 있으므로, 결과를 받은 뒤 현재 상태를 다시 확인합니다.

07. 계획 승인과 쟁점 추적

계획 해시·사용자 승인·지적 ID

계획 버전과 승인 무효화

서버는 계획의 줄바꿈 형식을 LF로 통일하고, 문서 앞뒤 공백을 정리한 뒤 끝에 줄바꿈 하나를 붙여 해시를 계산합니다. 본문의 조건·경로·정규식은 임의로 단순화하지 않습니다. Claude와 Codex의 확인값, 사용자의 승인값이 모두 현재 계획 해시와 같아야 구현을 시작합니다.

```
state == AWAITING_USER_APPROVAL
claude.ack == currentPlanSHA256
codex.ack == currentPlanSHA256
userApprovedSHA256 == currentPlanSHA256
```

계획 승인 대기 중 새 결정이나 근거가 들어오면 기존 확인·승인을 무효화하고 계획 회차를 올립니다. 범위 자체가 달라지면 scopeGeneration을 올려 대화와 세션을 분리하며, 이미 구현을 시작한 작업에는 새 worktree를 만듭니다. 같은 범위의 계획을 재검토하는 경우와 새로운 범위의 작업을 시작하는 경우를 구분했습니다.

지적 ID와 처리 상태

각 응답은 findings에 ID, 심각도, 설명, 근거, 처분을 담습니다. 다음 응답이 이전 ID를 빠뜨리면 서버가 거부합니다. 고치기로 한 지적을 근거 없이 “조치 없음”으로 바꾸는 경우에도 자동으로 통과시키지 않습니다. 사용자가 처분을 바꾸려면 해당 지적 ID를 명시한 결정을 남겨야 합니다.

처분	다음 단계에서 확인할 내용
AGREED_ACTION	고치기로 한 문제가 남아 있습니다. 최종 리뷰에서도 이 값이면 수정이 끝난 것으로 보지 않습니다.
RESOLVED_BY_FIX	수정으로 해결했다는 의미입니다. 보완·최종 리뷰 단계에서만 허용하며, 그보다 앞선 계획 검토 단계에서는 사용할 수 없습니다.
EXTERNAL_EVIDENCE	외부 동작을 확인할 근거가 더 필요합니다. 모델의 추측만으로 종결하지 않고 추가 근거를 기다립니다.
DEFERRED_OUT_OF_SCOPE	현재 승인 범위에서 다루지 않는 항목입니다. 후속 목록에 남겨 다음 주제로 넘길 수 있게 합니다.

해시로는 계획의 동일성을 확인합니다. 기술적 타당성은 코드와 실행 증거를 바탕으로 모델이 검토하고 사용자가 판단합니다.

08. CLI 실행 권한

단계별 도구·파일·네트워크 권한

단계	허용한 동작	차단한 동작
Claude 계획·개정	저장소 읽기, 코드 검색, 공개 문서 검색과 읽기	Edit·Write, 작업 파일 변경, 자동 공용 메모리 기록
Claude 구현·수정	승인된 worktree의 코드 읽기·수정, 허용된 테스트·빌드 도구	공용 Git 메타데이터 쓰기, shell 네트워크, 공개 웹 검색·읽기 도구
Codex 감사·리뷰	해당 worktree와 필요한 Git 메타데이터 읽기, 허용된 근거 조사	코드 쓰기, 승인 요청, 인증 파일 읽기, 임의 외부 연결
해시 확인 톤	계획 본문과 확인 규칙을 받아 짧은 JSON으로 응답	저장소 탐색 도구와 기존 계획 탐색 이력

설정·환경 변수 관리

Codex는 앱이 관리하는 별도 설정 폴더에서 실행합니다. 모델과 추론 강도는 호출 인자로 전달하고, 사용자 홈의 임의 provider·plugin·hook 설정은 그대로 복사하지 않습니다. 필요한 환경 변수만 전달하며, 로그에서는 인증값으로 보이는 문자열을 가립니다. 세션 인증에 필요한 파일과 모델이 도구로 읽을 수 있는 파일도 구분합니다.

Claude에는 계획 단계에서 쓰기 도구를 제공하지 않고, 구현 단계에서만 해당 worktree의 쓰기를 허용합니다. 구현용 Workflow 도구는 하위 실행이 부모의 도구 제한을 이어받는지 확인한 뒤 연결했습니다. 역할별 모델 설정과 구현 전용 설정을 따로 저장해, 계획 모델을 바꿔도 구현 설정이 유지되도록 했습니다.

프로젝트 지침 전달

독립 worktree에는 Git 추적에서 제외한 프로젝트 지침 파일이 없어, CLI에 원본 저장소의 코드 스타일과 작업 규칙이 전달되지 않았습니다. 서버가 worktree의 지침 파일을 먼저 확인하고, 없으면 원본 저장소의 지침 파일을 읽어 프롬프트에 넣도록 고쳤습니다. 여러 worktree에 파일 사본을 배포하지 않아, 사본마다 규칙이 달라지는 문제도 피했습니다.

로컬 운영 범위

한 대의 Mac에서 사용하는 로컬 도구로 설계했으며, 서버는 loopback 주소에서만 실행합니다. 첫 접속 토큰은 검증 후 HttpOnly 쿠키로 전환합니다. 여러 사용자가 인터넷으로 접속하는 서비스나, 에이전트가 독자적으로 외부 계정의 상태를 바꾸는 도구로는 운영하지 않습니다.

09. 파일 검증·메모리 반영·화면 갱신

본문 해시·저장 순서·이벤트 번호

파일 저장 순서와 기준 데이터

계획과 보고서는 본문의 SHA-256을 파일명으로 사용해 blob에 저장합니다. DB에는 경로와 해시를 기록하고, 사람이 읽기 편한 plan.md 등의 파일은 별도로 갱신합니다. 저장 순서는 blob 준비 → 현재 작업 확인 → DB 기록 → 읽기용 파일 교체입니다. DB 기록과 파일 교체 사이에 작업이 중단돼도, 다음 실행에서는 DB가 가리키는 blob을 읽습니다.

blob 준비

→ 세대·상태·취소 확인

→ DB에 경로·SHA 기록

→ plan.md 등 읽기용 파일 교체

다시 읽기: DB 경로 → 본문 SHA 재계산 → 저장 SHA와 대조

초기에는 DB에 저장된 해시만 확인하고 파일 본문을 그대로 읽어, 변조된 파일도 정상 결과로 사용하는 문제가 있었습니다. 파일을 읽을 때마다 실제 본문의 해시를 다시 계산하도록 고쳤습니다. 재개 프롬프트에 전달하는 계획 경로도 읽기용 별칭 대신 verifiedPath가 검증한 blob 경로로 바꿨습니다.

응답 검증 후 메모리 반영

모델에는 공용 메모리 폴더 전체에 대한 접근 권한을 주지 않습니다. 서버가 주제에 맞는 문서를 골라 전달하면, 모델은 memoryUpdates로 변경을 제안합니다. 서버는 역할별 경로, 읽기 시점과 현재 시점의 해시, 응답 계약과 작업 상태를 검사한 뒤 메모리에 반영합니다. 검증 전에 메모리를 쓰던 순서를 바꿔, 거부된 응답이나 취소된 결과가 다음 작업의 지침에 반영되지 않도록 했습니다.

SSE 재연결과 이벤트 중복 제거

타임라인은 주제별 sequence로 정렬합니다. 서버는 마지막으로 보낸 번호보다 큰 이벤트만 전송하고, 화면은 초기 조회와 실시간 이벤트가 겹쳐도 같은 번호를 기준으로 병합합니다. 브라우저 연결이 잠시 끊겨도 모델 작업을 실패로 처리하지 않고, 서버 기록에서 실행 상태를 다시 읽습니다. 요청에 실패해도 사용자 입력은 보존해 다시 보낼 수 있게 했습니다.

DB·파일·메모리를 하나의 분산 트랜잭션으로 묶는 대신, 기준이 되는 기록을 명확히 정했습니다. 중간에 실패해도 검증된 파일과 현재 상태를 기준으로 재개하도록 구성했습니다.

10. 실행 중단과 복구

중복 요청·취소·프로세스 종료·재시작

중복 요청과 실행 잠금

변경 요청은 Idempotency-Key와 본문을 함께 기록합니다. 같은 키와 본문이면 저장된 응답을 반환하고, 같은 키에 다른 본문이면 409로 거부합니다. 주제별 실행 잠금은 서로 다른 키로 같은 작업을 중복 시작하는 것도 막습니다. 요청 기록으로 HTTP 재전송을 처리하고, 실행 잠금으로 동시에 진행할 수 없는 작업을 제어합니다.

취소 후 응답 처리

CLI 호출 전 범위 세대와 기대 상태를 저장하고, 응답 직후 취소 신호·현재 세대·상태를 다시 대조합니다. 이전 범위에서 시작한 응답으로는 새 계획이나 메모리를 확정할 수 없습니다. 실행 중 새 decision이나 evidence가 들어오면 결과를 중단 기록으로 보존하되, 새 입력을 반영하지 않은 결과로 다음 단계를 진행하지 않습니다. 단순 note는 실행을 중단하지 않도록 구분했습니다.

프로세스 그룹 종료와 회수

PID 하나의 종료만으로 작업 전체가 끝났다고 판단하지 않습니다. 생성 시 PID·프로세스 그룹·명령·시작 시각을 기록하고, 중단 시 그룹에 SIGTERM을 보낸 뒤 남은 프로세스에는 SIGKILL을 보내 종료를 확인합니다. 재시작 때도 식별 정보가 일치하는 프로세스만 회수해, 재사용된 PID의 다른 작업을 종료하지 않도록 했습니다.

재현한 문제	개선한 처리
즉시 끝나는 프로세스에 긴 입력을 보내면 EPIPE로 서버 종료	stdin에 쓰기 전에 오류 핸들러를 등록하고, 성공·실패·취소 시 종료 처리를 하나의 경로로 통합했습니다.
두 번째 서버가 정상 실행 중인 첫 서버의 작업을 회수	포트 바인딩에 성공한 서버만 이전 실행을 회수하도록, 회수 시점을 onListen 이후로 옮겼습니다.
서버 종료 중 DB가 먼저 닫혀 실행 기록 마감 실패	새 실행 거부 → 실행 취소 → action 기록 마감 대기 → DB 닫기 순서로 바꿨습니다.
CLI가 결과 JSON을 냈지만 프로세스가 계속 남음	최종 결과 수신 후 120초 안에 종료되지 않으면 종료를 요청하고, 결과 수신 후 종료한 경우로 구분해 기록합니다.

종료 대기 시간에도 상한을 두었습니다. 실행 마감 대기는 15초, 서버의 최종 강제 종료는 25초입니다. 재시작 시 진행 중이던 action은 중단으로 기록하고, resume_state에 저장된 단계부터 재개합니다. 종료가 지연되더라도 진행 중인 작업을 성공으로 처리하지 않습니다.

11. 리뷰본·커밋·원격 코드 검증

HEAD·변경 해시·Git tree·원격 OID

리뷰 시점의 코드 저장

먼저 HEAD와 변경 파일의 SHA-256을 기록합니다. 변경 파일 해시에는 기준 commit, 정렬한 경로, 파일 권한·내용, 삭제 여부와 심볼릭 링크 대상을 포함하며, 추적되지 않은 파일도 검사합니다. git diff의 텍스트가 같아도 이 값들이 다르면 같은 결과로 보지 않습니다.

이어서 임시 GIT_INDEX_FILE로 작업 폴더를 Git tree 객체에 저장합니다. 실제 index와 작업 파일은 변경하지 않고, 객체가 정리되지 않도록 참조를 남깁니다. 최종 리뷰에는 직전 tree와 현재 tree의 차이를 전달해, 수정된 부분과 그 영향을 받는 코드를 다시 검토하도록 합니다.

검사 시점	검사하는 값과 실패 시 처리
구현·수정 직후	브랜치와 HEAD가 구현 시작 기준에서 바뀌지 않았는지 확인합니다. 응답 교정 중 코드가 바뀔 수 있으므로 마지막 교정 호출 뒤에도 다시 확인합니다.
커밋 직전	READY_TO_DELIVER 상태, 리뷰본의 HEAD·변경 해시, 사용자 선택 경로를 대조합니다. 기존 staged 변경, 실행 가능한 Git hook 등 전달 결과를 바꾸는 조건은 거부합니다.
커밋 직후	실제 commit의 부모, 변경 경로, 파일 내용·권한·삭제·링크가 리뷰본과 같은지 확인합니다. 검증에 실패하면 식별값을 남기고 멈춥니다.
push 이후	원격 브랜치가 방금 확인한 commit OID를 가리키는지 다시 조회합니다. 로컬 명령의 성공 메시지만으로 완료하지 않습니다.

Git 작업의 결과 미확정 처리

commit·push는 외부 상태가 변경된 뒤 서버 응답이 끊길 수 있습니다. 이 경우 일반 실행 실패와 구분해 결과 미확정으로 기록합니다. 확인한 OID를 기준으로 부모·경로·내용·원격 상태를 다시 대조한 뒤 완료를 확정합니다. 검증되지 않은 commit은 자동 reset으로 삭제하지 않고 보존합니다.

주제별 worktree와 범위 세대

주제마다 detached worktree에서 계획을 세우고, 구현 승인 후 전용 브랜치를 만듭니다. 범위가 바뀌면 세대 번호가 포함된 새 브랜치와 worktree를 사용합니다. 이전 계획·리뷰·커밋 기록을 보존해 각 범위에서 어디까지 완료했는지 추적할 수 있습니다. 커밋과 push는 각각 사용자의 결정 후 서버가 실행합니다.

12. 계획 개정 최적화

입력 이력 축소와 계획 부분 수정

개정 이력 누적과 출력 한도

계획 개정은 현재 계획에 감사 지적을 반영하는 작업입니다. 기존 세션을 이어 쓰면 폐기된 개정 전문과 탐색 기록까지 입력에 다시 포함됐습니다. 운영 기록에서는 개정 입력이 약 834K 토큰, 1회 소요 시간이 약 45분까지 늘었습니다. 긴 계획·지적 처분·메모리 전문을 한 응답에 담다가 64K 출력 한도를 넘긴 사례도 있었습니다.

새 세션과 필수 자료 전달

개정마다 새 세션을 만들고, 현재 계획·감사 결과·사용자 결정과 근거를 명시적으로 전달했습니다. 중복된 agent_output payload는 타임라인에서 제외하고 중요한 사용자 입력과 최근 기록을 남겼습니다. 계획 해시를 확인하는 ACK도 도구 없는 별도 턴으로 분리해, 저장소를 다시 탐색하지 않도록 했습니다.

planEdits를 이용한 부분 수정

```
planEdits: [
  { find: "기존 계획의 정확한 문장",
    replace: "지적을 반영한 새 문장" }
]
```

각 find가 현재 본문에 정확히 1번 있는지 검사
→ 순서대로 교체 → 계획 계약 검사 → 새 SHA 계산

find가 없으면 다른 버전을 기준으로 작성한 수정일 수 있고, 여러 번 나오면 수정 위치가 모호하므로 적용을 거부합니다. 빈 배열은 수정 사항이 없는 개정으로 허용하고, 전체 구조를 바꿀 때는 계획 전문으로도 응답할 수 있게 했습니다. 이전 계획 전문이 필요할 때는 타임라인의 잘린 메시지 대신 저장된 계획 파일을 직접 읽습니다.

전후 기록	관찰한 값
계획 개정 입력	약 834K → 135K 토큰, 약 84% 감소
개정 1회 소요 시간	약 45분 → 16분, 약 64% 감소
측정 조건	동일 Mac에서 같은 대형 계획을 다룬 변경 전후 각 1회 기록입니다. 새 세션·부분 수정·입력 정리가 함께 적용된 결과입니다.

새 세션에는 이전 대화가 없으므로, 현재 지적과 결정이 프롬프트에서 누락되지 않아야 합니다. 입력을 줄이면서도 지적 ID 승계와 계획 계약 검사는 유지했습니다. 측정값은 해당 계획의 변경 전후 기록이며, 모든 작업에 적용되는 평균 절감률로 해석하지 않았습니다.

13. 중단 결과 재사용

결정 후 재개와 수정 회차 관리

결정 대기 중 결과 보존

계획이나 수정 결과에 사용자 결정이 필요한 항목이 있으면 작업을 중단합니다. 초기 재시도 로직은 저장된 결과를 활용하지 못해 모델을 다시 호출했고, 일부 경로에서는 첫 계획부터 다시 시작했습니다. 개정 결과가 이미 저장돼 있는데도 사용자 결정 후 재개할 때 개정을 다시 실행하는 문제가 있었습니다.

멈춘 지점	결정 후 재사용 조건과 다음 행동
계획·개정	저장된 본문 또는 planEdits가 현재 계약을 만족하는지 확인합니다. 사용자 결정이 있으면 저장본을 계획으로 확정하고 감사 또는 종결 확인으로 넘어갑니다.
코드 수정	저장된 수정 결과가 대상 리뷰 이후에 생성됐고, 이후 사용자 결정·지적 처리·기준 HEAD가 조건에 맞으면 수정 호출 없이 최종 리뷰로 진행합니다.
최종 리뷰	리뷰 이후 코드가 그대로이고 사용자의 결정으로 남은 조건이 해소됐다면, 저장된 리뷰로 전달 준비 상태를 확정합니다. 코드가 바뀌면 다시 리뷰합니다.

수정 완료 시점의 회차 기록

호출 시작 시 수정 회차를 차감하면, 사용 한도나 응답 실패로 결과를 받지 못한 작업도 수정 1회로 계산됩니다. 이를 결과를 검사하고 최종 리뷰로 넘어가는 시점에 기록하도록 바꿨습니다. 확인된 결함이 남으면 두 번째 수정까지 자동으로 진행하고, 이후에도 남으면 해당 ID를 제시한 뒤 사용자 결정을 기다립니다.

추가 쟁점에 대한 재개정 제한

첫 종결 확인에서 새 지적이 나오면 두 번째 개정을 한 번 더 허용해 반영하도록 했습니다. 이전 개정과 두 번째 개정의 ID를 합쳐 이미 검토한 쟁점인지 판별합니다. 추가 개정 후에도 새 쟁점이 남으면 사용자 판단을 기다립니다. 이처럼 추가 검토 횟수를 제한해 자동 검토가 끝없이 반복되지 않도록 했습니다.

저장 결과를 재사용할 때는 파일 존재 여부뿐 아니라, 대상 리뷰·계획 이후에 생성됐는지, 현재 코드·계획과 일치하는지, 새 결정으로 중단 사유가 해소됐는지 확인합니다.

14. 세션 분리와 주제별 병렬 실행

역할별 세션과 worktree별 실행 큐

계획·구현·리뷰 세션 분리

구현은 계획 세션을 fork하는 대신 새 세션에서 시작합니다. 승인된 계획·전문·결정·프로젝트 지침을 전달하고, 이후 수정은 구현 세션을 이어 씁니다. 코드 리뷰도 계획 감사와 분리된 Codex 세션에서 시작합니다. 첫 리뷰에는 승인 계획·최종 쟁점 처분·구현 보고·사용자 근거를 전달하고, 수정 후 재검토는 같은 리뷰 세션에서 진행합니다.

```
리뷰 세션 재사용 조건
현재 scopeGeneration == 저장된 scopeGeneration
현재 planEpoch == 저장된 planEpoch
현재 planSHA256 == 저장된 planSHA256
```

리뷰 세션은 SQLite에 저장해 서버 재시작 후에도 이어 쓰되, 범위나 승인 계획이 달라지면 새로 시작합니다. 재개 턴에는 마지막으로 전달한 이벤트 번호 이후의 기록과 계획 SHA·검증된 파일 경로를 전달합니다. 전달 번호는 호출이 반환된 뒤 기록해, 실패한 턴에서 전달되지 않은 이벤트가 다음 입력에서 빠지지 않도록 했습니다. 기존 세션을 찾지 못하면 전체 프롬프트로 새 세션을 만듭니다.

주제별 설정 폴더와 실행 큐

처음에는 모든 Codex 실행이 하나의 config.toml을 덮어썼습니다. 작업 A가 설정을 준비한 뒤 실행되기 전에 작업 B가 설정을 바꿀 수 있어, 설정 준비부터 CLI 종료까지 전 구간을 직렬화했습니다. 설정 충돌은 막았지만 서로 다른 주제의 감사도 앞선 작업이 끝날 때까지 기다려야 했습니다.

worktree 경로의 해시로 주제별 설정 폴더를 만들고 config.toml을 각각 저장합니다. 같은 worktree는 큐로 직렬 실행하고, 서로 다른 worktree는 세마포어로 전체 동시 실행 수를 제한합니다. 기본 상한은 2입니다. 세션·캐시 등 공유 상태는 연결해 재개를 유지하되, 공유 폴더를 준비하는 구간만 별도로 직렬화했습니다.

동시 실행 검증

서로 다른 두 주제가 동시에 실행되는지, 상한을 넘는 세 번째 실행은 대기하는지, 같은 worktree의 두 번째 턴은 앞선 턴이 끝난 뒤 시작하는지 테스트했습니다. 서로 다른 주제 사이의 대기를 줄이는 변경이며, 토큰 사용량이나 전체 완료 시간의 개선 여부는 별도로 측정하도록 했습니다.

15. 범위 밖 변경 규칙과 diff 검사

승인 경로·변경 규칙·실제 코드 대조

경로·변경 형태·개수 제한

Swift 6 전환 중 다른 단계에서 다루는 파일에 표기 한 줄을 추가해야 하는 상황이 반복됐습니다. 파일 전체의 수정을 허용하면 동작 변경까지 섞일 수 있고, 매번 사용자 결정을 기다리면 작업이 끊깁니다. 이를 제한적으로 허용하기 위해 승인 계획에 경로·변경 형태·파일 수·변경 구간 수를 명시하는 허용 오차 규칙을 추가했습니다.

계획: scopePaths + rules[]
 규칙: id / paths / hunk / tokens / maxFiles / maxHunks
 결과: toleranceLedger[{ ruleId, file, note }]

실제 변경 파일 → 승인 경로 박인가?
 → 원장 행 → 규칙 경로 → diff 조건 → 개수 상한

insert-token은 기존 줄을 유지한 채 허용 토큰만 삽입했는지 확인합니다. annotation-only는 양쪽에서 지정 토큰을 제거했을 때 같은지 봅니다. any는 더 넓은 변경을 허용하므로 경로와 개수 상한으로 제한합니다. 새 파일은 any만 허용하고, 바이너리·권한 변경도 단순 토큰 삽입으로 통과하지 못하게 했습니다.

Swift 문자열·주석·보간식 구분

처음의 토큰 비교만으로는 문자열 값에 같은 단어를 넣은 변경을 구분하기 어려웠습니다. Swift의 일반·여러 줄·raw 문자열, 주석을 추적하는 렉서를 추가하고, 문자열 보간의 괄호와 중첩 문자열은 프레임 스택으로 추적하도록 보강했습니다. diff의 잘린 줄만 보고 문맥을 추측하지 않도록 기존 commit의 파일 내용으로 앞선 상태도 재구성합니다.

코드: func a() {} → nonisolated func a() {}
 문자열: "func a() {}" → "nonisolated func a() {}"

같은 단어가 추가돼도 문자열 값의 변경은
 코드 토큰 삽입 규칙으로 허용하지 않습니다.

규칙 위반 교정과 HEAD 재검사

구현·수정 결과를 받은 뒤, 리뷰에 앞서 서버가 실제 diff와 원장을 대조합니다. 규칙을 위반하면 같은 세션에 한 번 교정을 요청하고, 다시 위반하면 사용자 결정을 기다립니다. 교정 중 생성한 commit으로 diff가 가려지지 않도록, 코드를 변경할 수 있는 마지막 호출 뒤에도 HEAD를 검사합니다. 규칙 밖의 후속 작업은 별도 목록으로 남깁니다.

이 검사는 문법 차이와 승인 범위 준수 여부를 확인합니다. 표기 변경이 런타임 동작에 미치는 영향과 Swift 동시성 계약의 타당성은 리뷰와 단계별 컴파일·실행 검증에서 확인합니다.

16. 사용 한도 오류의 자동 재시도

예약·취소·재시작 복원·소요 시간 기록

한도별 재시도 시각과 횟수

오류 메시지에서 session·weekly·monthly-spend 등의 한도 종류와 해제 시각을 파싱합니다. 자동 재시도가 가능한 한도는 해제 시각에 90초의 여유를 더해 예약하고, 시각을 읽지 못하면 30분 뒤로 예약합니다. 지출 한도는 자동 재시도하지 않습니다. 같은 한도 오류가 반복되면 연속 자동 재시도를 3회로 제한합니다.

상황	예약 처리 방식
수동 retry 또는 새 action	기존 예약을 취소합니다. 사용자가 새 작업을 시작한 뒤 타이머가 중복 실행하지 않도록 합니다.
사용자 stop	예약 취소 사실을 DB에 남깁니다. 서버가 재시작해도 취소한 예약을 되살리지 않습니다.
범위 변경	예약과 해당 범위의 재시도 상태를 초기화합니다. 이전 계획의 실패를 새 세대로 가져가지 않습니다.
서버 재시작	시도 횟수·마지막 실행 시각·예약 시각·취소 여부를 읽어 복원합니다. 기존 예약이 있으면 예약 시각을 유지합니다.
타이머 실행 직전	여전히 FAILED인지, 실행·범위 변경·전달 잠금이 없는지 다시 확인한 뒤 retry를 호출합니다.

실패 시각 기준의 예약 복원

“오전 8시 50분에 해제”라는 오류를 오후에 복원할 때 현재 시각을 기준으로 파싱하면, 해제 시점이 다음 날로 밀릴 수 있습니다. 실패 기록의 시각으로 해제 시점을 구하고, 남은 대기 시간만 현재 시각과 비교하도록 고쳤습니다. 테스트에서는 시간을 직접 제어해 날짜·시간대·재시작·취소 상황을 실제 대기 없이 검증합니다.

도구 실행 시간과 모델 호출 시간

사용량 이벤트에 입력·캐시·출력 토큰과 전체 시간을 기록하고, Claude의 API 시간과 두 CLI의 도구 시작·종료 이벤트로 측정한 시간도 추가했습니다. Codex의 전체 시간에서 도구 실행 시간을 빼도 대기 시간은 남으므로, 이를 모델 추론 시간으로 단정하지 않습니다. 도구 실행이 긴 작업과 모델 응답이 긴 작업을 구분하는 자료로 사용합니다.

예약 기능의 동작은 테스트로 확인했습니다. 사람의 대기 시간과 실제 완료 시간이 얼마나 줄었는지는 같은 모델·추론 설정의 운영 기록을 모아 비교하도록 했습니다.

17. iOS 앱 Swift 6 전환 적용

단계별 적용 결과와 실패 사례

작업별 승인·검토·전달 기록

초기 적용에서는 Rx 직접 의존 제거, 진단 기준선 작성, 공통 기반·연결 계층, 데이터 모델, 네트워크·지도·화면 이동, 공용 UI·WebKit, 기능 모듈 전환의 7개 작업을 완료했습니다. 각 작업은 별도 worktree에서 진행하고 계획 승인 전 검토 항목 122개를 ID로 추적했습니다. 7개 작업 모두 리뷰 뒤 commit과 원격 작업 브랜치를 대조해 전달했습니다.

실패 사례와 판단 오류	적용한 처리와 확인 결과
로그 출력만으로 빌드 성공 판단	tee 이후의 출력만 확인하는 대신, xcodebuild의 실제 종료값을 기록하도록 검토 항목에 명시했습니다. 종료값을 확인하기 전에는 해당 지적을 종결하지 않았습니다.
중단된 빌드가 남긴 파일을 완료 결과로 재사용	종료값·파일 SHA·재시도 번호·단계가 포함된 완료 기록을 요구했습니다. 다음 측정이 어느 실행을 이어받았는지 확인했습니다.
WebKit 객체 생성 테스트만 통과	객체 해제 시 exit 139를 재현하고 관련 동작 테스트 7개를 다시 통과시켰습니다. 객체 생성 시점의 성공과 해제 시점의 안전성을 구분해 검증했습니다.
OS 전체 fatal 로그를 앱 실패로 집계	기기 UDID와 앱 프로세스를 고정해 다시 확인했습니다. OS 전체에서 잡힌 7건과 앱 프로세스의 fatal 0건을 구분했습니다.
데이터 모델 전환 중 후속 단계의 설계까지 범위 확대	추가 설계 3건은 후속 작업으로 넘기고 승인한 경로만 커밋했습니다. 다음 단계에서 검토할 근거도 함께 보존했습니다.

실행 원장을 다시 확인한 결과, 중형 기능·채팅 SDK 경계, 디코딩 격리 보안, 지도·미디어, 매물 광고·등록, 앱 서비스의 후속 작업까지 전달을 마쳤습니다. 초기 7개를 포함한 12개 작업의 commit과 push OID가 일치했습니다. 앱의 나머지 영역을 전환하는 다음 작업은 구현 중이었습니다.

서버 자동 검사와 운영 판단

서버는 계획 해시, 지적 ID, 작업 상태, 파일 SHA, 커밋·원격 OID를 반복해서 검사합니다. iOS 빌드 종료값의 의미, WebKit 해제 오류의 원인, 앱 로그의 대상, 단계별 제품·코드 범위는 중재 과정에서 원본 증거를 확인해 판단했습니다. 모델의 요약이 실행 결과와 다르면 실제 로그와 코드를 기준으로 바로잡았습니다. 최종 앱 검증에서는 29개 번들의 테스트 3,291개와 화면 이동 계약 189개를 통과했고, 빌드·설치·실행 후 앱 fatal은 0건이었습니다. 이는 당시 Swift 6 전환 코드의 검증 결과이며, 개발 브랜치 병합이나 이후 전체 전환의 완료를 의미하지는 않습니다.

18. 세션별 사용량 측정

계획 개정에서 입력·시간 감소 관찰 · 전체 작업 기준 토큰 절감률은 미측정

변경 전 → 변경 후 → 판단

항목	변경 전	변경 후	판단
계획 개정	기존 세션 이력 유지 계획 전체 출력 약 834K 토큰 · 45분	새 세션·입력 정리 계획 부분 수정 약 135K 토큰 · 16분	입력·시간 감소 관찰 입력 약 84%, 시간 약 64% 감소. 동일 계획의 전후 각 1회 기록.
첫 구현	계획 세션 fork 이전 탐색 이력 승계 입력 약 545K 토큰	새 구현 세션 시작 계획·결정·지침 전달 입력 약 92K 토큰	입력 감소 기록 다른 단계의 실행이라 동일 조건 비교가 아님. 전체 토큰 절감률은 확인하지 못함.
코드 리뷰	계획 감사 세션에서 코드 리뷰 진행	계획 감사와 별도인 리뷰 전용 세션 사용	사용량 개선 여부 판단 불가 작업·범위·리뷰 단계가 달라 직접 비교할 수 없음.
병렬 실행	공용 설정 파일 사용 모든 주제를 직렬 실행	주제별 설정 폴더 분리 다른 주제는 병렬 실행	동작 검증 완료 동시 실행·상한을 확인함. 대기 시간 감소량은 미측정.

K는 1,000 토큰입니다. 계획 개정은 같은 Mac에서 새 세션·부분 수정·입력 정리를 함께 적용한 결과입니다. 개별 변경의 효과
나 모든 작업의 평균 절감률은 확인하지 않았습니다.

리뷰 세션 사용량 기록 | 전후 비교가 아닌 참고 기록

이전 방식의 최종 리뷰와 새 세션의 첫 리뷰는 작업과 읽어야 할 코드 양이 다릅니다. 아래 수치가 커지거나 작아졌다는
이유만으로 세션 분리의 개선·악화를 판단하지 않았습니다.

실행 기록	입력 토큰	전체 시간
이전 방식의 최종 리뷰	1,938,779	581초
전용 세션의 첫 코드 리뷰	4,176,872	781초
전용 세션을 재개한 최종 리뷰 1회차	2,805,814	854초
같은 전용 세션의 최종 리뷰 2회차	1,093,479	595초

입력 합계에는 캐시 토큰도 포함되므로 입력 토큰만으로 세션 효율을 단정하지 않았습니다. 이후에는 같은 모델·추론 설정과
리뷰 단계에서 변경 규모·캐시 사용량을 함께 기록하고, 도구 실행과 대기 시간을 구분하도록 측정 방식을 바꿨습니다.

상태·승인·코드 검사는 유지합니다. 중단 결과 재사용·변경분 전달·재시도 예약으로 불필요한 호출을 줄입니다.

19. 검증 범위와 설계 제약

테스트 파일 20개·테스트 403건 통과 · TypeScript 검사·Vite 빌드 통과

실패·복구 시나리오 검증

검증 대상	검증한 계약
상태·승인·지적	허용되지 않은 전이, 이전 계획 승인, 빠진 지적 ID, 근거 없는 처분 변경을 거부합니다. 결정 후 저장된 결과로 이어지는 경로도 확인합니다.
파일·메모리	파일 본문 변조, 세대가 바뀐 뒤 결과 저장, 검증 실패 응답의 메모리 반영, 동시 메모리 변경을 검사합니다.
실제 자식 프로세스	즉시 종료와 EPIPE, 취소, 부모 종료 뒤 남은 자식, 최종 결과 뒤 종료 지연을 재현합니다. 프로세스 식별값이 다르면 회수를 거부합니다.
실제 임시 Git 저장소	리뷰 후 코드 변경, 선택 범위 밖 파일, commit 내용 불일치, 교정 중 승인 없이 생성한 commit, 허용 오차 위반·교정·재실패를 확인합니다.
세션·동시 실행	주제별 설정 분리, 동시 실행 상한, 같은 주제의 직렬 처리를 확인합니다. 리뷰 세션은 범위·계획 변경에 따른 폐기와 재개를 검증합니다.
예약·화면	테스트에서 시간을 직접 제어해 한도 해제·예약 복원·취소·상한을 검사하고, UI 테스트로 이벤트 병합과 설정·재시도 표시를 확인합니다.

자동 테스트의 한계

자동 테스트에서는 정해진 응답을 반환하는 테스트용 어댑터를 사용합니다. 403개 테스트는 서버의 상태·권한·복구·전달 규칙을 검증한 결과로, 실제 Claude·Codex 응답의 품질이나 서비스 가용성을 보장하지 않습니다. iOS 앱의 화면·컴파일·런타임은 각 전환 작업에서 별도로 검증했습니다.

구조별 제약과 운영 부담

SQLite와 로컬 파일은 한 대의 Mac에서 설치·복구하기 쉽지만 다중 사용자 원격 서비스에 맞춘 구조는 아닙니다. worktree와 결과 파일 보존으로 디스크 사용량이 늘어, 종료된 작업의 빌드 산출물만 정리하고 검증 로그는 남기는 기능을 추가했습니다. 새 세션에서는 판단에 필요한 자료가 프롬프트에서 누락되지 않아야 합니다.

16개 상태와 여러 승인·해시 검사는 구현 복잡도를 높이지만, 재실행이 필요한 작업과 저장 결과로 재개할 작업, 사용자 결정이 필요한 상황을 코드로 구분할 수 있게 합니다. 이 구분을 유지하면서 사람이 반복하던 복사·대조·재시도 작업을 줄이는 방향으로 개선해 왔습니다.